Shader Report

Ali Gasimov, Rashid Huseynov, Nazrin Azizli

May 2026

Contents

1 Introduction **2**

2 Methodology **2**
2.1 Procedural Terrain with Fractal Brownian Motion . . . 2
2.2 Height-Based Color Ramp and Surface Detail . . . 3
2.3 Blinn-Phong Lighting Model . . . 4
2.4 Post-Processing Pipeline . . . 4
2.4.1 Gaussian Blur . . . 5
2.4.2 Bloom Effect . . . 5
2.5 Camera System . . . 5
2.6 Browser-Based Scene Editor . . . 5
2.6.1 Architecture . . . 6
2.6.2 View Mode . . . 6
2.6.3 Object Management . . . 6
2.6.4 Transform Controls . . . 6
2.6.5 Terrain Controls . . . 6
2.6.6 Scene-Wide Lighting Controls . . . 7
2.6.7 Status Bar . . . 7

3 Implementation **7**
3.1 Shader Files . . . 7
3.2 Source Modules . . . 8
3.3 Tests . . . 9

4 Evaluation and Results **9**
4.1 Terrain Generation . . . 9
4.2 Lighting . . . 10
4.3 Post-Processing . . . 10
4.4 Scene Editor . . . 10
4.5 Performance . . . 11

5 Discussion **11**
5.1 Limitations . . . 11
5.2 Conclusion . . . 11

Abstract

This project implements a GLSL and js based 3D renderer and editor, allowing you to insert 3D visuals into websites easily with a set of prebuilt shaders. It uses GLSL shaders, instead of Three.js built in materials, letting users quickly use their own GLSL shaders. The editor allows you to edit the environment directly from the browser, allowing quick iteration on 3D environments. Default shaders, such as vertex displacement, phong lighting and Gaussian blur.

1 Introduction

A shader is a small, specialized program that runs on a computer's graphics processing unit (GPU) to determine the color, lighting, and special effects of pixels and vertices in 3D computer graphics whereas threejs is a popular, lightweight, cross-browser JavaScript library used to create and display animated 3D computer graphics in a web browser, without needing proprietary plugins.

Real-time 3D graphics on the web have become increasingly accessible thanks to the WebGL API and libraries such as Three.js [6], which expose the GPU's programmable pipeline to JavaScript developers. At the heart of any real-time renderer are *shaders*: small programs written in GLSL (OpenGL Shading Language) that run in parallel on the GPU to determine the color and position of every pixel on screen. Shaders are split into two main types: *vertex shaders*, which process each vertex of a 3D mesh and compute its final screen position, and *fragment shaders* (also called pixel shaders), which run for each pixel covered by a triangle and output a final color.

Three.js is a high-level JavaScript library that wraps WebGL, providing abstractions for scenes, cameras, meshes, lights, and materials. While Three.js ships with a rich set of built-in materials such as `MeshPhongMaterial` and `MeshStandardMaterial`, it also exposes a `ShaderMaterial` class that lets developers inject raw GLSL code directly into the rendering pipeline. This project makes exclusive use of `ShaderMaterial`, bypassing all Three.js built-in shading in favor of hand-written GLSL.

Procedural terrain generation is a technique by which geometry and texture are synthesized algorithmically rather than authored by hand. A common approach is to use noise functions, particularly *fractal Brownian motion* (fBm), which layers multiple octaves of smooth noise at increasing frequencies and decreasing amplitudes to produce natural-looking surfaces [3]. Further realism can be added through *domain warping*, where the input coordinates to the noise function are themselves displaced by a second noise evaluation, producing flowing, folded structures reminiscent of rivers and mountain ridges [4].

Post-processing refers to a class of image-space effects applied after the main scene has been rendered into an off-screen buffer. Rather than operating on 3D geometry, post-processing shaders treat the rendered image as a texture and manipulate it as a 2D image. Common effects include Gaussian blur, depth of field, and bloom – the glow that appears around bright light sources in both photography and the human eye. Bloom is typically implemented in three steps: extracting bright pixels above a luminance threshold, blurring the extracted image, and compositing the result back on top of the original scene.

This project builds all of these systems from scratch as a structured codebase, with clearly separated modules for core rendering, scene management, shader materials, post-processing, and camera control. The following sections describe the methodology behind each subsystem, the implementation, and the results observed during development and testing.

2 Methodology

2.1 Procedural Terrain with Fractal Brownian Motion

The terrain is generated entirely on the GPU inside the vertex shader. A flat plane of 256 by 256 vertex segments is created in JavaScript, then each vertex is displaced upward along the Y axis by a height value computed in GLSL. This avoids the need to precompute or store a heightmap on the CPU.

The core height function is fractal Brownian motion over 2D simplex noise. For n octaves, the fBm sum is defined as:

$$h(p) = \sum_{i=0}^{n-1} a_i \cdot \text{simplex2}(f_i \cdot p + o_i) \tag{1}$$

where $a_i = 0.5^i$ is the amplitude (halved per octave), $f_i = 2^i$ is the frequency (doubled per octave), and o_i is a per-octave offset vector to decorrelate successive layers. The result is normalized by the sum of amplitudes so the output stays in the range $[-1, 1]$.

To produce more dramatic mountain ridges, the terrain also supports a *ridge fBm* variant. Instead of accumulating noise directly, it accumulates the folded value $1 - |n|$, sharpening valleys into ridgelines:

$$r(p) = \sum_{i=0}^{n-1} a_i \cdot w_i \cdot (1 - |\text{simplex2}(f_i \cdot p)|) \tag{2}$$

where w_i is a running weight clamped to $[0, 1]$ that causes high-amplitude octaves to suppress subsequent contributions, emphasizing the sharpest features.

Domain warping is applied by computing a 2D fBm pair and using it to offset the terrain coordinates before the main fBm evaluation:

$$\text{warp}(p) = \begin{pmatrix} \text{fbm2}(p) \\ \text{fbm2}(p + c) \end{pmatrix}, \quad h = \text{fbm2}(p + s \cdot \text{warp}(p)) \tag{3}$$

where c is a constant offset vector to decorrelate the two warp components and s is a uniform controlling warp strength. This produces the characteristic folded and fluid appearance of real eroded terrain.

A sine-wave blend mode is also provided to simulate animated water surfaces. Three overlapping sine waves with different directions, frequencies, and speeds are summed and blended with the fBm result via a configurable uniform:

$$h_{\text{final}} = \text{mix}(h_{\text{fbm}},\ h_{\text{sine}},\ \texttt{u_sineBlend}) \tag{4}$$

Because the terrain is displaced in the vertex shader, surface normals are not automatically recomputed by Three.js and must be approximated using finite differences. For a small step ϵ, the normal at point p is:

$$N = \text{normalize}\left(\begin{pmatrix} h(p - \epsilon_x) - h(p + \epsilon_x) \\ 2\epsilon / A \\ h(p - \epsilon_z) - h(p + \epsilon_z) \end{pmatrix}\right) \tag{5}$$

where A is the amplitude uniform. This gives a consistent world-space normal at every vertex that is passed to the fragment shader for per-pixel lighting.

2.2 Height-Based Color Ramp and Surface Detail

The terrain fragment shader assigns color based on the interpolated height value passed from the vertex shader. Seven color bands are defined, spanning from deep ocean at the bottom to snow caps at the top. Between adjacent bands, a smooth linear interpolation is applied based on where the height falls within that band's range:

A procedural micro-detail overlay is blended on top of the color ramp using high-frequency simplex noise to simulate surface texture variation such as grass blades and rock graininess. Water surfaces below a threshold height receive an extra animated specular highlight to simulate sunlight glinting on moving water. Exponential fog is applied at the end of the fragment shader based on camera distance:

$$f = e^{-d_{\text{fog}} \cdot \text{dist}} \tag{6}$$

where d_{fog} is a configurable density uniform and dist is the world-space distance from the camera to the fragment.

Height range	Zone
0.00 – 0.15	Deep water
0.15 – 0.28	Shallow water
0.28 – 0.42	Sand / beach
0.42 – 0.62	Lowland grass
0.62 – 0.78	Highland
0.78 – 0.90	Bare rock
0.90 – 1.00	Snow cap

Table 1: Height-based terrain color zones in the fragment shader.

2.3 Blinn-Phong Lighting Model

The lighting model is the Blinn-Phong reflectance model [1], which decomposes reflected light into three additive terms. The **ambient** term models indirect environmental illumination:

$$I_a = k_a \cdot C_{\text{ambient}} \cdot C_{\text{surface}} \tag{7}$$

The **diffuse** term models scattering from a rough surface according to Lambert's cosine law:

$$I_d = k_d \cdot C_{\text{light}} \cdot C_{\text{surface}} \cdot \max(\hat{N} \cdot \hat{L},\ 0) \tag{8}$$

The **specular** term models mirror-like reflections using the half-vector $H = \text{normalize}(L + V)$ between the light direction and the view direction:

$$I_s = k_s \cdot C_{\text{light}} \cdot \Big(\max(\hat{N} \cdot \hat{H},\ 0)\Big)^{\alpha} \tag{9}$$

where α is the shininess exponent and k_a, k_d, k_s are material coefficients. The total illumination is the sum of all three terms:

$$I = I_a + I_d + I_s \tag{10}$$

All calculations are performed per-pixel in the fragment shader using world-space positions and normals passed as varyings from the vertex shader. A debug mode uniform isolates individual lighting components at runtime via keyboard shortcuts (keys 1 through 4).

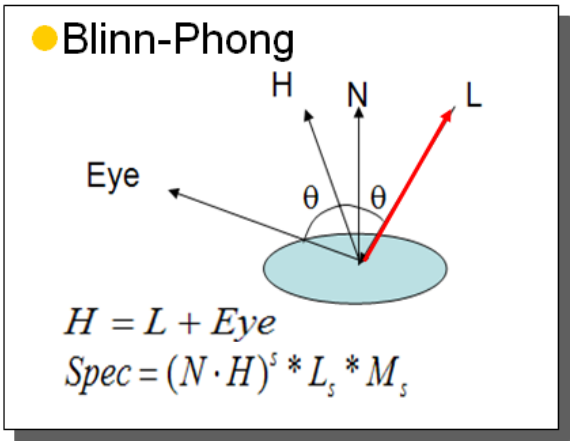


Figure 1: Blinn-Phong

2.4 Post-Processing Pipeline

The post-processing system is built around an off-screen render target architecture. Each frame, the main scene is rendered into a `WebGLRenderTarget` rather than directly to the canvas. This texture is then passed through a chain of full-screen quad passes before the final result is written to the screen.

2.4.1 Gaussian Blur

The blur effect uses a two-pass separable Gaussian kernel. The separable property of the Gaussian function allows the 2D convolution to be decomposed into a horizontal pass followed by a vertical pass, each requiring only $O(k)$ texture samples instead of $O(k^2)$ for a naive 2D kernel [2]. The 5-tap kernel weights used are:

$$w = [0.2270,\ 0.1946,\ 0.1216,\ 0.0541,\ 0.0162] \tag{11}$$

Multiple passes of this blur can be applied sequentially to increase the effective blur radius without widening the kernel. Pass count and per-sample pixel spacing are configurable.

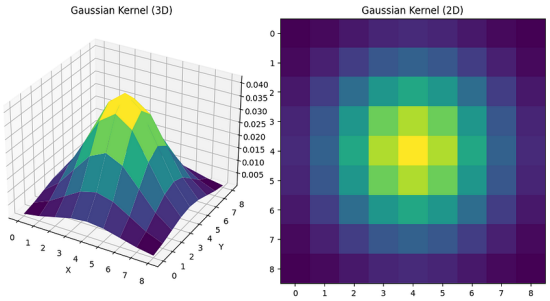


Figure 2: GaussianBlur

2.4.2 Bloom Effect

The bloom effect is implemented as a three-step pipeline [5]:

1. **Brightness extraction**: A fragment shader outputs only pixels whose luminance exceeds a configurable threshold. Luminance is computed using standard perceptual weights:

   $$L = 0.2126 \cdot R + 0.7152 \cdot G + 0.0722 \cdot B \tag{12}$$

   A `smoothstep` transition around the threshold avoids hard edges in the extracted image.

2. **Blur**: The extracted bright pixels are passed through the Gaussian blur described above, spreading the glow outward from bright areas.

3. **Composite with Reinhard tone mapping**: The blurred bloom texture is added on top of the original scene, then Reinhard tone mapping compresses the result back into the displayable $[0, 1]$ range:

   $$C_{\text{out}} = \frac{C_{\text{scene}} + I \cdot C_{\text{bloom}}}{C_{\text{scene}} + I \cdot C_{\text{bloom}} + \vec{1}} \tag{13}$$

   where I is a configurable bloom intensity uniform.

2.5 Camera System

The camera uses the browser's Pointer Lock API to capture raw mouse delta values, enabling smooth first-person look-around. Movement is driven by WASD keys for horizontal movement, Q and E for vertical movement, and Shift for a sprint multiplier. Velocity is smoothed each frame using linear interpolation toward the target velocity, producing damped, fluid movement rather than instantaneous direction changes.

2.6 Browser-Based Scene Editor

To support rapid iteration without restarting the application, a browser-based scene editor panel is implemented as a drop-in overlay rendered on top of the WebGL canvas. The editor communicates directly with live JavaScript objects — the `SceneManager`, `Lights`, and `TerrainMaterial` instances — and propagates changes into GLSL uniforms on the same frame they are made, so the visual result is immediately visible.

2.6.1 Architecture

The editor is encapsulated in a single `SceneEditor` class that takes a `SceneManager` reference and a `THREE.Scene` at construction time. All DOM construction is performed programmatically; no separate HTML template is required. A single `injectStyles()` function inserts a scoped stylesheet into `document.head` on first use, making the editor fully self-contained and safe to drop into any existing page.

The panel is organized into six collapsible sections, each of which can be individually opened or closed to reduce visual clutter. A minimize button collapses the entire panel to its title bar when the full canvas view is needed.

2.6.2 View Mode

Four view mode buttons switch the `uViewMode` uniform on the Phong material at runtime, isolating the **Combined**, **Ambient**, **Diffuse**, or **Specular** lighting components. This mirrors the keyboard shortcuts (keys 1–4) already present in `SceneManager` and provides a GUI equivalent accessible without memorizing bindings.

2.6.3 Object Management

The *Add Object* section lets the user select a geometry primitive (Box, Sphere, Cylinder, Torus, or Plane), specify a size, and pick a base color. Clicking *Add to Scene* constructs the corresponding `THREE.Geometry`, clones the active Phong `ShaderMaterial` so the new mesh shares the same GLSL shaders and uniform bindings, and inserts the mesh into the Three.js scene graph at a random position. If the Phong material has not yet finished loading its GLSL sources asynchronously, the editor falls back to a `THREE.MeshPhongMaterial` so the object still appears immediately.

All editor-added objects are tracked in an internal list rendered in the *Objects* section. Clicking a list entry selects it, populates the *Transform* section with its current values, and highlights the entry. Individual objects can be removed with a per-row delete button, or all editor objects can be cleared at once.

2.6.4 Transform Controls

When an object is selected, the *Transform* section exposes numeric inputs for its X/Y/Z **position**, **rotation** (in radians), and **scale**. Each field updates the corresponding property on `mesh.position`, `mesh.rotation`, or `mesh.scale` on every `input` event, providing sub-frame feedback as the user types or drags.

2.6.5 Terrain Controls

The *Terrain* section maps directly onto the uniform setters exposed by `TerrainMaterial`:

- **Y Offset** — vertical translation of the terrain mesh.
- **Amplitude** — peak displacement height of the fBm, range 0–20.
- **Frequency** — base spatial frequency fed to the noise function.
- **Octaves** — integer count of fBm layers, range 1–8.
- **Warp** — domain warp strength s from Equation 3.
- **Ridge** — blend factor between standard fBm and ridge fBm.
- **Water** — sine-wave blend factor `u_sineBlend` from Equation 4.
- **Fog** — exponential fog density d_{fog} from Equation 6.
- **BG Color** — scene background color, applied to `scene.background`.

Each control uses a range slider with a live numeric readout. Default values match the initial uniform values set in `TerrainMaterial`, so the editor state is consistent with the scene on load.

2.6.6 Scene-Wide Lighting Controls

The *Scene* section provides controls that affect the global lighting state managed by the `Lights` instance:

- **Ambient color** — a color picker whose value is converted from hex to a normalized RGB triplet and applied via `Lights.setAmbientColor()`.
- **Ambient intensity** — a scalar multiplier in the range 0–2. Because the underlying `Lights` class stores ambient color as a plain `THREE.Vector3` rather than a separate intensity, the editor maintains the base color and intensity as independent state variables and recombines them on every change: $C_{\text{ambient,final}} = C_{\text{base}} \times I$. Both controls therefore work independently without interfering with each other.
- **Shininess** — updates the `uShininess` uniform on the Phong material directly, controlling the specular exponent α.
- **Toggle Debug Cube** — shows or hides the rotating Phong-shaded box that `SceneManager` adds to the scene as a material test target.

Because `SceneManager.update()` pushes `Lights.getUniforms()` into `PhongMaterial` on every frame, changes made through the ambient controls take effect within one frame with no additional wiring required.

2.6.7 Status Bar

A persistent status bar at the bottom of the panel displays a short confirmation message after each editor action (e.g. "Added Sphere_3", "View mode → Ambient") that fades back to "ready" after three seconds. This provides lightweight feedback without modal dialogs or console output.

3 Implementation

3.1 Shader Files

- `shaders/common/uniforms.glsl`: Declares the four uniforms shared across all shaders: `uTime`, `uResolution`, `uLightPos`, and `uCameraPos`.
- `shaders/common/lighting.glsl`: Defines the reusable GLSL helper functions `ambientLight`, `diffuseLight`, and `specularLight` that implement the Blinn-Phong terms. Prepended to any shader that needs lighting.
- `shaders/noise/simplex.glsl`: Implements 2D and 3D simplex noise and the fBm family: `simplex2`, `simplex3`, `fbm2`, `fbm3`, `ridgeFbm`, and `fbmWarp`.
- `shaders/noise/perlin.glsl`: Implements classic 2D and 3D Perlin noise (`perlin2`, `perlin3`) as an alternative noise primitive.
- `shaders/terrain/vertex.glsl`: Computes per-vertex height via fBm, ridge fBm, domain warping, and sine-wave blending. Displaces the vertex along Y and approximates the world-space normal via finite differences.
- `shaders/terrain/fragment.glsl`: Applies the height-based color ramp, micro-detail overlay, Blinn-Phong lighting, animated water specular, and exponential fog.
- `shaders/phong/vertex.glsl`: Transforms vertex positions and passes world-space position, normal, and UV coordinates to the fragment shader.
- `shaders/phong/fragment.glsl`: Applies Blinn-Phong lighting using the helpers from `lighting.glsl`. Supports `uViewMode` to isolate ambient, diffuse, or specular components for debugging.

- `shaders/post/blur.glsl`: Reference GLSL for the separable Gaussian blur (active implementation is inlined in `BlurPass.js`).
- `shaders/post/bloom.glsl`: Reference GLSL for the brightness extraction and composite passes (active implementation is inlined in `BloomPass.js`).

3.2 Source Modules

- `src/main.js`: Entry point. Instantiates the `App` class on the canvas element.
- `src/App.js`: Top-level integration class. Wires all subsystems and drives the per-frame loop: update camera, update scene uniforms, render scene to off-screen target, run post-processing to screen.
- `src/core/Renderer.js`: Thin wrapper around `THREE.WebGLRenderer`. Handles pixel ratio, size updates, and disposal.
- `src/core/Sizes.js`: Listens for window resize events and broadcasts reactive width, height, and pixel ratio to all subscribers.
- `src/core/Time.js`: Drives the animation loop via `requestAnimationFrame` and emits a `tick` event each frame with elapsed and delta time.
- `src/camera/Camera.js`: Wraps `THREE.PerspectiveCamera` and updates the aspect ratio on resize.
- `src/controls/CameraController.js`: Implements free-fly movement using WASD, Q, E, and Shift with smoothed velocity and Pointer Lock mouse look.
- `src/controls/OrbitControls.js`: Fallback orbit camera for cases where Pointer Lock is unavailable.
- `src/scene/SceneManager.js`: Coordinates the scene. Owns the `THREE.Scene` instance, initializes terrain and the Phong debug mesh, and calls per-frame updates on all scene objects.
- `src/scene/SceneEditor.js`: Self-contained browser-based editor panel. Constructs its own DOM and stylesheet, and exposes six collapsible sections: View Mode, Add Object, Objects, Transform, Terrain, and Scene. Communicates directly with the live `SceneManager`, `Lights`, and `TerrainMaterial` instances, pushing changes into GLSL uniforms on the same frame they are made. Tracks all editor-added objects and provides per-object selection, transform editing, and deletion.
- `src/scene/Terrain.js`: Creates the `THREE.PlaneGeometry` terrain mesh and owns its lifecycle. Exposes `update(elapsed)`, `setPosition()`, and `setResolution()`.
- `src/scene/Lights.js`: Manages light position and color, and exposes `getUniforms()` to push current values into shader uniforms each frame.
- `src/rendering/ShaderMaterial.js`: Base class for all custom materials. Creates a `THREE.ShaderMaterial` and updates the shared uniforms (`uTime`, `uCameraPos`, `uResolution`) each frame.
- `src/rendering/PhongMaterial.js`: Extends `ShaderMaterial` for Phong lighting. Provides a static async `create()` factory that fetches GLSL sources, prepends `lighting.glsl`, and returns a ready-to-use material.
- `src/rendering/TerrainMaterial.js`: Extends `ShaderMaterial` for the terrain. Fetches and concatenates noise and terrain GLSL files at load time. Exposes a `whenReady()` promise and per-frame uniform setters.
- `src/postprocessing/PostProcessor.js`: Owns the scene render target and the `EffectComposer`. Exposes `bloomEnabled` and `blurEnabled` toggles (B and L keys) and a single `render()` call per frame.

- `src/postprocessing/EffectComposer.js`: Holds an ordered list of post-processing passes and chains them so the output of each pass feeds into the next.
- `src/postprocessing/BloomPass.js`: Implements the three-step bloom pipeline: brightness extraction, Gaussian blur to a dedicated intermediate render target, and Reinhard composite to the output.
- `src/postprocessing/BlurPass.js`: Implements the two-pass separable Gaussian blur. Runs a horizontal pass then a vertical pass, repeating for a configurable number of iterations.
- `src/utils/MathUtils.js`: JavaScript-side math helpers including `lerp`, `smoothstep`, and normal matrix computation.
- `src/utils/Noise.js`: JavaScript-side noise helpers for any CPU-side procedural work.

3.3 Tests

All tests use Vitest [7] and run in Node.js with Three.js mocked, requiring no browser or GPU.

- `tests/Camera.test.js`: Verifies the `Camera` wrapper exposes a `THREE.PerspectiveCamera` instance and responds to resize events.
- `tests/CameraController.test.js`: Checks that key bindings update velocity vectors correctly and that `destroy()` removes all event listeners cleanly.
- `tests/BlurPass.test.js`: Verifies construction, render target creation, and the `setSize` method.
- `tests/BloomPass.test.js`: Verifies extraction and composite render targets are created with correct dimensions and that the `enabled` flag bypasses processing correctly.
- `tests/EffectComposer.test.js`: Checks that passes are added in order and executed in sequence.
- `tests/OrbitControls.test.js`: Verifies fallback orbit camera construction and basic behavior.

4 Evaluation and Results

4.1 Terrain Generation

The fBm terrain with domain warping produces naturally varied heightfields that visually resemble eroded landscapes. Increasing the octave count from 1 to 6 progressively adds fine surface detail while preserving the large-scale shape. The ridge fBm mode introduces sharp mountain ridgelines at little additional cost since it reuses the same simplex noise evaluations. The sine-wave blend mode produces convincing animated water ripples, with the blend uniform allowing smooth transitions between land and ocean surface modes at runtime.

The finite-difference normal approximation using $\epsilon = 0.01$ produces smooth, artifact-free normals consistent with the displaced geometry. Because the same `heightAt` function is used for both displacement and normal computation, the normals correctly reflect all active blend modes including domain warping and sine blending.

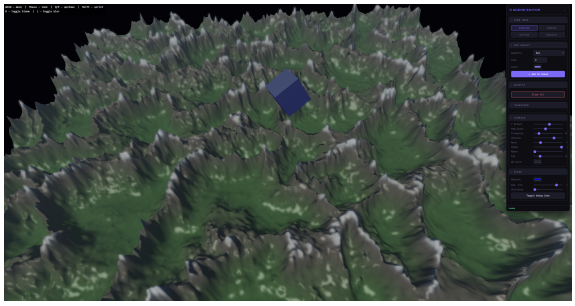


Figure 3: Produced Terrain

4.2 Lighting

The Blinn-Phong implementation produces the expected visual results: a smooth diffuse gradient across curved surfaces, a tight specular highlight whose size scales correctly with the shininess exponent, and a constant ambient floor preventing fully black shadows. The `uViewMode` debug uniform proved useful during development for isolating specular artifacts from diffuse or ambient issues.

4.3 Post-Processing

The bloom effect makes bright surfaces such as the water specular highlight visually glow. Reinhard tone mapping correctly prevents over-exposure, keeping the brightest areas from clipping to pure white.

A significant bug was identified and fixed during development: the internal blur step inside `BloomPass` was originally writing to `null` (the screen) instead of a named intermediate render target. This caused the blurred image to appear on the canvas before the composite step had run, making the bloom overlay look frozen and out of sync with the animated scene. The fix was to allocate a dedicated `_blurOutputRT` so the compositor always reads fresh data.

The separable Gaussian blur produces smooth, artifact-free softening. Running multiple iterations increases the effective blur radius without widening the kernel, keeping performance cost predictable.

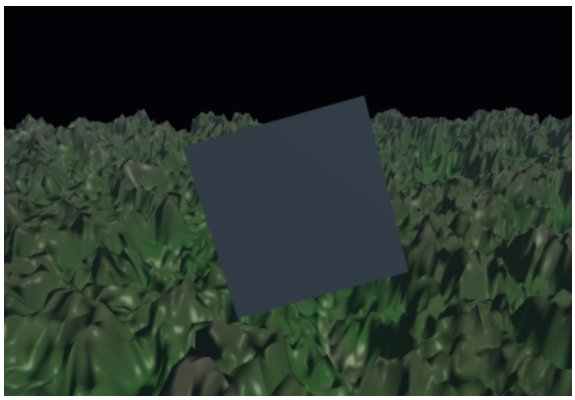


Figure 4: Gaussian Blur Result

4.4 Scene Editor

The editor panel renders correctly across Chrome and Firefox with no impact on frame time, as all DOM updates are triggered only by user interaction rather than the per-frame render loop. Terrain uniform changes — amplitude, frequency, warp, octaves — take effect on the next vertex shader invocation, which is the following frame, giving sub-frame visual feedback.

Three bugs were identified and resolved during editor development. First, the ambient color picker and intensity slider both silently no-oped because the editor was reading from the return value of `Lights.getUniforms()`, which returns raw `THREE.Vector3` instances rather than Three.js uniform wrapper objects. The fix was to call `Lights.setAmbientColor()` directly, letting the existing per-frame uniform propagation in `SceneManager.update()` carry the change into the shader. Second, the intensity slider referenced a non-existent `ambientIntensity` property; this was resolved by maintaining the base color and intensity as separate editor-side state and combining them at the point of application. Third, newly added objects were being inserted into the scene graph twice due to a duplicate `scene.add()` call, which doubled the draw call cost and caused object-list state to desync from the scene.

After these fixes, the editor's ambient controls behave as expected: the color picker and intensity slider are independently editable and their combined effect is visible immediately in both the main Phong material and the ambient-only view mode.

4.5 Performance

Because all heavy computation runs on the GPU, CPU load is minimal and JavaScript is responsible only for uniform updates and draw calls. At 256 by 256 terrain segments, approximately 66,000 vertex shader invocations run per frame, each evaluating multiple octaves of simplex noise plus a domain warp, and this runs smoothly at 60 fps in Chrome on typical hardware. The separable blur achieves $O(k)$ texture samples per pixel per pass rather than $O(k^2)$ for a naive 2D convolution, keeping post-processing cost manageable as pass count increases.

5 Discussion

5.1 Limitations

Vertex-level normal approximation. Normals are computed per vertex and linearly interpolated across triangles, so the lighting does not capture normal variation at sub-vertex scale. At lower terrain resolutions this causes visible faceting in the specular highlight. A higher-quality alternative would compute the normal per fragment by sampling the height function inside the fragment shader, at the cost of additional noise evaluations.

No shadow casting. The implementation does not support shadow mapping. All surfaces receive the full lighting calculation regardless of occlusion. Adding shadows would require an additional render pass from the light's perspective to produce a depth map, followed by a shadow lookup in the terrain fragment shader.

Single light source. The Phong model supports one directional light. Extending to multiple lights would require either a uniform array of light parameters or a deferred shading approach using G-buffer render targets.

Global bloom. The bloom effect applies to any pixel above the luminance threshold regardless of whether it is a physically emissive surface. A more accurate approach would use a separate emissive render target to mark only surfaces that should produce glow.

Editor state not serialized. Changes made through the scene editor are held in memory only. Refreshing the page resets all uniform values to their coded defaults. Persisting editor state would require serializing the current uniform values and terrain parameters to `localStorage` or a query string.

5.2 Conclusion

This project demonstrates that a complete real-time 3D rendering system including procedural geometry, physically inspired lighting, post-processing effects, and a live browser-based editor can be built from first principles in the browser using Three.js as a minimal scaffold and custom GLSL for all visual computation. The modular architecture separating noise, lighting, terrain, materials, post-processing, and editor makes the system extensible: new shaders or effects can be added by following the existing patterns without modifying unrelated code. The combination of fBm noise, domain warping, and height-based coloring produces convincing natural-looking terrain,

while the Blinn-Phong model and bloom pipeline bring the scene to life with dynamic lighting. The scene editor removes the edit-save-reload cycle from the creative workflow, allowing terrain shape, lighting mood, and scene composition to be explored interactively in real time.

Code Availability

The full project source is available at: `https://github.com/sonicisastorm/shader-project`

References

[1] James F. Blinn. "Models of Light Reflection for Computer Synthesized Pictures". In: *ACM SIGGRAPH Computer Graphics* 11.2 (1977), pp. 192–198.

[2] Paul Heckbert. *Filtering by Repeated Integration*. Tech. rep. SIGGRAPH 1986 proceedings. ACM, 1986.

[3] David S. Ebert et al. *Texturing and Modeling: A Procedural Approach*. 3rd. Morgan Kaufmann, 2002.

[4] Inigo Quilez. *Domain Warping*. Accessed May 2026. 2002. URL: `https://iquilezles.org/articles/warp/`.

[5] Masaki Kawase. *Frame Buffer Postprocessing Effects in DOUBLE-S.T.E.A.L (Wreckless)*. GDC 2003 presentation. 2003.

[6] Ricardo Cabello et al. *Three.js – JavaScript 3D Library*. Accessed May 2026. 2024. URL: `https://threejs.org`.

[7] The Vite Team. *Vitest – A Vite-Native Unit Test Framework*. Accessed May 2026. 2024. URL: `https://vitest.dev`.